Fundamentals of Programming using Python • First Year, FinTech

Solutions — TP 01

Python: What It Is and How It Speaks

Worked answers to the 8 exercises

Dr. Brahim Zirari

Yahia Farès University of Medea

in linkedin.com/in/brahimzirari · Google Scholar

October 2026

Solution 1. The timeline – Without looking back at the chapter, write the year of each of these five events: (a) Guido van Rossum starts Python; (b) the first public release; (c) Python 3.0 appears; (d) Python 2 stops being supported; (e) the version you should install today, and its number. (a) 1989 — December, as a holiday hobby project. (b) 1991 — version 0.9.0. (c) 2008 — Python 3.0, a deliberate break with Python 2. (d) 1 January 2020 — Python 2 reached its end of life. (e) Python **3.14**, released October 2025 and still receiving bug fixes.

Solution 2. Why Python, and the glue – (a) Give four reasons why Python is used so widely, and for each one a single sentence on why it matters to a FinTech student in particular. (b) Explain in one sentence what a *glue language* is, then explain why calling Python slow is only half true. (a) Any four of the six reasons in section 2. Model answers: **Free and open source** — a student can install the same tool a bank uses, at no cost. **Most used language** — the skill transfers to any employer. **Large community** — you are never stuck alone on an error. **Ready-made libraries** — pandas reads a bank statement in one line, so you spend your time on the finance, not on the file format.

(b) A glue language is one that is easy to use to connect, coordinate and control programs written in other languages. Python itself executes slowly, but the libraries that do the heavy work — NumPy, pandas — are written in compiled C. You write Python and the machine runs C, so the speed that matters is C's.

Solution 3. Your first program – Write a file that prints three lines: the text `Banking fee report`, the year `2026` as a number, and the VAT of 19% on a fee of 1 250 DZD. Put a comment above the third line saying which rate is applied.

```
print("Banking fee report")
print(2026)
# VAT rate on banking fees, 2026
print(1250 * 0.19)
```

Output

```
Banking fee report
2026
237.5
```

Remark: The calculation inside the brackets is done first and only the result is printed. Note that `237.5` prints with a decimal point: a calculation involving `0.19` produces a `float`.

Solution 4. Variables and an f-string – A sales agent is paid a commission of 2.5% on what he sells. Last month he sold 1 340 000 DZD. Store the sales figure and the commission rate in two named variables, compute the commission in a third, then print one line reading `1340000 DZD of sales earns 33500.00 DZD commission` — the amount shown with exactly two decimals.

```
sales = 1340000
rate = 0.025
commission = sales * rate
print(f"{sales} DZD of sales earns "
      f"{commission:.2f} DZD commission")
```

Output

```
1340000 DZD of sales earns 33500.00 DZD commission
```

Remark: Three separate jobs on three separate lines: store the inputs, compute, report. Written this way, changing the commission rate means editing one character, and the printed line follows by itself. The two strings on the last two lines are joined by Python into one line of output — the usual way to keep a long `print` inside the width of the screen.

Solution 5. Two kinds of division – An invoice of 7 450 DZD is to be split between 4 people. Display, on three lines: the exact share, the whole number of dinars each person pays, and the remainder left over. Then verify the identity $a = b \times (a \,//\, b) + (a \,\%\, b)$ by printing its right-hand side.

```
print(7450 / 4)
print(7450 // 4)
print(7450 % 4)
print(4 * (7450 // 4) + 7450 % 4)
```

Output

```
1862.5
1862
2
7450
```

Remark: `/` always gives a float. `//` throws the fractional part away. `%` gives what is left over,

which here is the 2 DZD that cannot be split four ways. The fourth line reconstructs the original amount exactly, which is what makes the identity useful as a check.

Solution 6. Powers and rounding – A bank card PIN is 4 digits, each one of `0`–`9`. The matching online account uses a password of 8 characters drawn from the 62 letters and digits. Display, on three lines: (a) the number of possible PINs; (b) the number of possible 8-character passwords; (c) if a machine can test one PIN every 0.35 seconds, the number of hours needed to try them all, rounded to 2 decimals.

```
print(10 ** 4)
print(62 ** 8)
seconds = 10 ** 4 * 0.35
print(round(seconds / 3600, 2))
```

Output

```
10000
218340105584896
0.97
```

Remark: Each extra character multiplies *the possibilities rather than adding to them, which is what `**` expresses. The two numbers are worth comparing: every PIN can be tried in under an hour, while the 8-character password has more than 218* trillion *possibilities. This is why a PIN is useless without the bank's three-attempt limit, and why the password protects the account while the PIN protects only the card. `round(x, 2)` keeps two decimals; without it the third line would print `0.9722222222222222`.*

Solution 7. Types and conversion – Print the type of each of these four values: `250000`, `0.035`, `"250000"` and `7 / 1`. Then, in two further lines, add 8 to the text `"42"` to obtain `50`, and join the text `"Total: "` to the number `1250`.

```
print(type(250000))
print(type(0.035))
print(type("250000"))
print(type(7 / 1))
print(int("42") + 8)
print("Total: " + str(1250))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'float'>
50
Total: 1250
```

Remark: Two results deserve attention. `"250000"` is a `str`, not an `int`: the quotes decide, not the contents. And `7 / 1` is a `float`, because `/` always produces one. The last two lines show the rule for `+`: between two numbers it adds, between two strings it joins, and between one of each it refuses — so one side must be converted.

Solution 8. Read the error – (a) A program fails with `NameError: name 'capitl' is not defined` at line 3. What is wrong, and how do you fix it? (b) A tutorial contains the line `print "Total: ", total`. What is wrong with it, what is the correct Python 3 line, and what does it tell you about the tutorial? (c) A user types `250000` into `input()` and the program then computes `capital * 0.035`. It fails with a `TypeError`. Why, and what single change repairs it? (a) `capitl` is a misspelling of `capital`: the name was never created, so Python has nothing to look up. Fix the spelling on line 3. A `NameError` is almost always a typo or a variable created *after* the line that uses it.

(b) In Python 3, `print` is a *function*, so its arguments must be inside round brackets. Written without them the line is a `SyntaxError` and nothing runs at all. The correct form is `print("Total: ", total)`. A tutorial using the old form was written for Python 2, unsupported since 1 January 2020 — find a more recent source.

(c) `input()` always returns a **string**, so `capital` holds the text `"250000"`, and text cannot be multiplied by `0.035`. Wrap the input: `capital = float(input(...))`. Note that `"250000" * 3` would *not* have raised an error — it would have printed the text three times. The silent version is the more dangerous one.